

Continuous Integration pipeline using GitHub Actions.

Step 1. Create a folder with name “.github” in your project repository. Now create another nested folder with the name “workflows”. All the workflows for our GitHub Actions would be stored as “yaml” file here.

The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying the project structure: DEMO-PROJECT-REPO, .github, workflows, CI-Workflow.yml, app.py, and requirements.txt. The main editor shows the CI-Workflow.yml file with a single line: `1 Press (CTRL + I) Chat with BLACKBOX.AI / (CTRL + L) Share Code`. The terminal panel at the bottom shows the following commands and output:

```
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo\.github> cd workflows
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo\.github\workflows> dir
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo\.github\workflows> cd ..
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo\.github> cd ..
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo> dir
```

The output of the `dir` command in the workflows directory is:

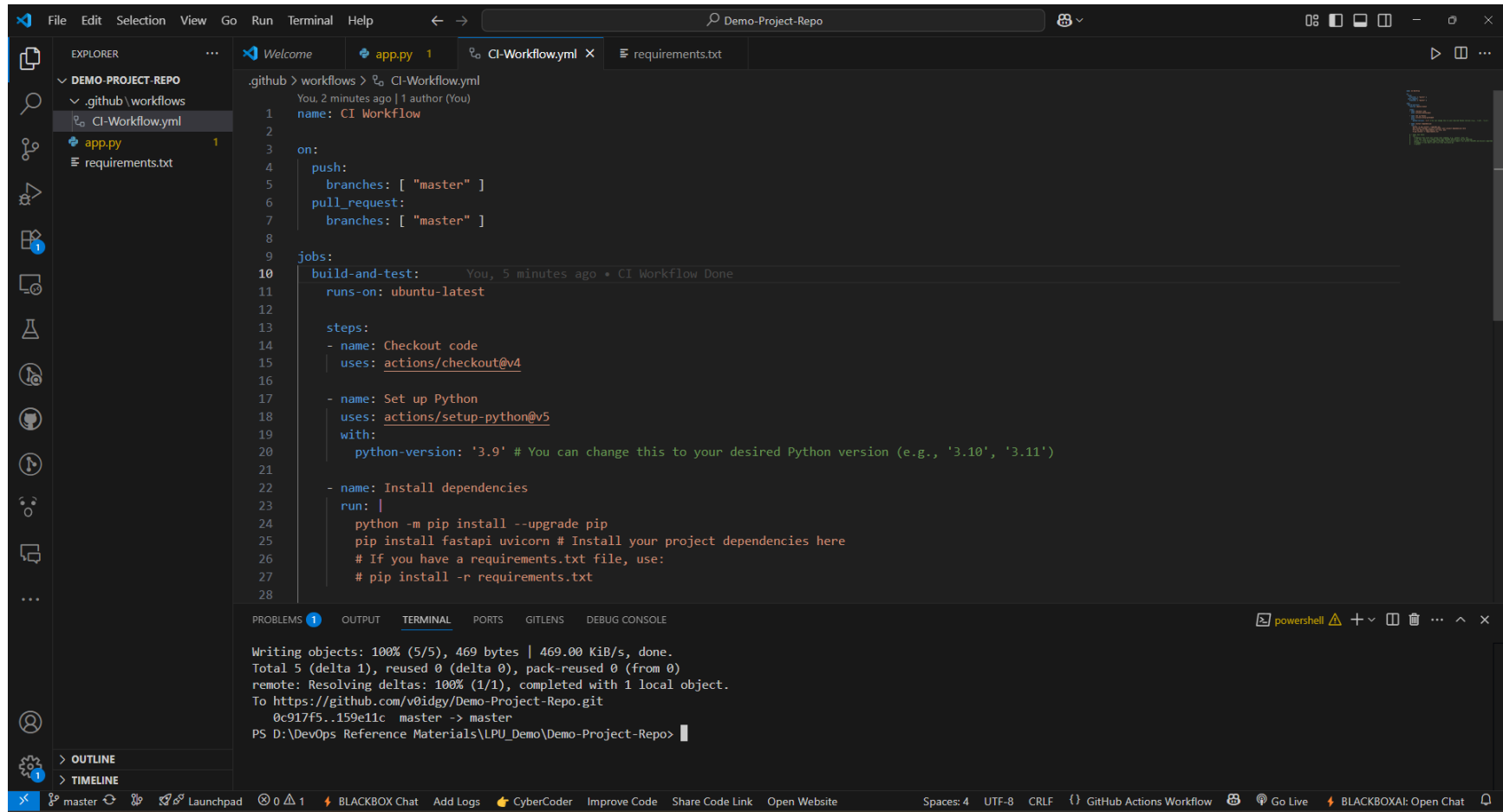
Mode	LastWriteTime	Length	Name
d----	25-06-2025 23:26		workflows

The output of the `dir` command in the root directory is:

Mode	LastWriteTime	Length	Name
d----	25-06-2025 23:26		.github
-a---	25-06-2025 23:25	125	app.py
-a---	25-06-2025 21:46	18	requirements.txt

The status bar at the bottom shows the current branch as master, the file encoding as UTF-8, and the file type as CRLF. It also displays the GitHub Actions Workflow file and the BLACKBOX.AI chat interface.

Step 2. Now write your Continuous Integration workflow in your YAML file mentioning all the jobs, steps, events, etc.



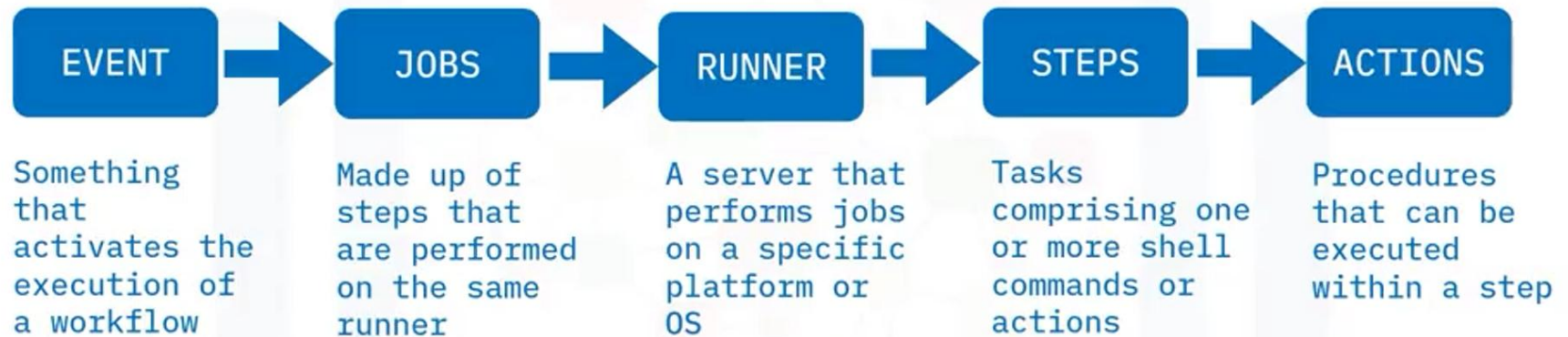
The screenshot shows the Visual Studio Code interface with a GitHub Actions workflow file open. The Explorer on the left shows the project structure: DEMO-PROJECT-REPO, .github/workflows, CI-Workflow.yml, app.py, and requirements.txt. The main editor displays the content of CI-Workflow.yml, which is a GitHub Actions workflow for building and testing the application. The workflow is triggered on push to the master branch or pull requests to the master branch. It consists of a single job named 'build-and-test' that runs on the 'ubuntu-latest' runner. The job has three steps: 'Checkout code' using 'actions/checkout@v4', 'Set up Python' using 'actions/setup-python@v5' with the Python version set to '3.9', and 'Install dependencies' which runs a series of pip commands to install dependencies from requirements.txt. The terminal at the bottom shows the output of the workflow run, indicating that the objects were written successfully and the dependencies were installed.

```
1 name: CI Workflow
2
3 on:
4   push:
5     branches: [ "master" ]
6   pull_request:
7     branches: [ "master" ]
8
9 jobs:
10  build-and-test:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v4
16
17      - name: Set up Python
18        uses: actions/setup-python@v5
19        with:
20          python-version: '3.9' # You can change this to your desired Python version (e.g., '3.10', '3.11')
21
22      - name: Install dependencies
23        run: |
24          python -m pip install --upgrade pip
25          pip install fastapi uvicorn # Install your project dependencies here
26          # If you have a requirements.txt file, use:
27          # pip install -r requirements.txt
28
```

Writing objects: 100% (5/5), 469 bytes | 469.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/v0idgy/Demo-Project-Repo.git
0c917f5..159e11c master -> master
PS D:\DevOps Reference Materials\LPU_Demo\Demo-Project-Repo>

Explanation

Workflow composition



Initial setup

```
.github
├── workflows
│   ├── ci.yaml
│   ├── cd.yaml
│   └── publish.yaml
```

Place workflow .yaml files

name: CI Workflow

- **Purpose:** This is the name of your workflow, which will appear in the GitHub Actions tab of your repository. It provides a human-readable title for your pipeline.

on:

- **Purpose:** This section defines the events that trigger the workflow to run.
 - **push::** The workflow will run whenever code is pushed to the specified branches.
 - **branches:** - main: Specifically, it triggers when changes are pushed to the main branch.
 - **pull_request::** The workflow will also run when a pull request is opened, synchronized, or reopened that targets

the specified branches.

- *branches:* - main: It triggers for pull requests opened against the main branch.
- **In summary:** This setup ensures that your CI pipeline runs automatically on every code change to main and on every proposed change via a pull request to main, allowing for early detection of issues.

Event examples

```
on:
  pull_request:
    types: [opened, reopened]
    branches:
      - master
```

- Runs whenever a pull request is opened or reopened against the master branch

```
on:
  push:
    branches:
      - main
```

- Runs whenever a push is made to the main branch including from merging a pull request

```
on:
  release:
    types: [publish]
```

- Runs whenever a release is published

jobs:

- **Purpose:** A workflow is composed of one or more jobs. This section defines these jobs. Each job runs in a separate virtual machine or container.
 - *build-and-test::* This is the name of your single job in this workflow. You can have multiple jobs (e.g., lint, deploy)

that run in parallel or sequentially.

Jobs

- Each workflow can have multiple jobs
- Jobs are named whatever you want
- Jobs run in parallel unless dependencies specified using 'needs' keyword
- Each job contains:
 - Runner
 - Services (optional)
 - Steps

```
jobs:  
  build:  
    runs-on: ubuntu-latest  
    steps:  
  
  publish:  
    runs-on: ubuntu-latest  
    needs: build  
    steps:
```

-

runs-on: ubuntu-latest

- **Purpose:** This specifies the type of runner (the virtual machine environment) where the job will execute. *ubuntu-latest* means it will run on the latest version of Ubuntu Linux. Other options include *windows-latest* or *macos-latest*.

Runner examples

```
jobs:
  build:
    runs-on: ubuntu-latest
```

This job runs on the latest version of Ubuntu

```
jobs:
  build:
    runs-on: ubuntu-22.04
```

This job runs on Ubuntu 22.04

```
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
```

This job runs inside a Python 3.9-slim Docker container (Debian 11) on top of the latest Ubuntu distribution

•

steps:

- **Purpose:** A job contains a sequence of tasks called steps. Each step executes a command or an action.
 - - name: Checkout code
 - **Purpose:** This step uses the actions/checkout@v4 action, which is a pre-built GitHub Action that checks out your repository code into the runner's environment. This is typically the first step in almost all workflows, as it makes your code available for subsequent steps.
 - - name: Set up Python

- **Purpose:** This step uses the `actions/setup-python@v5` action. It sets up a specific version of Python in the runner environment.
- **with:** `python-version: '3.9'`: This line specifies that Python version 3.9 should be installed and made available. You can change this to match your project's Python version.
- - **name:** Install dependencies
 - **Purpose:** This step runs shell commands to install your project's dependencies.
 - **run:** `|`: The `|` indicates a multi-line string, allowing you to execute several commands.
 - `python -m pip install --upgrade pip`: Upgrades the pip package installer.
 - `pip install fastapi uvicorn`: Installs FastAPI and Uvicorn, which are the core dependencies for your project based on the provided code.
 - `# If you have a requirements.txt file, use: and # pip install -r requirements.txt`: These are commented-out suggestions. If your project has a `requirements.txt` file listing all dependencies, you would uncomment and use that command instead.

Steps

- Steps can have an optional name (`name:`) displayed in the report
- Steps can have an optional ID (`id:`) to refer to other steps
- Steps have either an action (`uses:`) or shell commands (`run:`)
- Steps can have environment variables (`env:`)
- You can have as many steps as needed in a job

```
steps:

- name: Run unit tests with nose
  - id: testing
  - run: nosetests
  - env:
    - DATABASE_URI: "redis://redis:6379"
    - FLASK_APP: service:app

```

This structure provides a clear, automated process to ensure your code builds correctly, dependencies are installed, contributing to a robust development workflow.

Step 3. Now commit all the changes and push it to your repository. After pushing the file we could see the workflow running in GitHub Actions.

The screenshot displays the GitHub Actions interface for a repository named 'v0ldgy / Demo-Project-Repo'. The top navigation bar includes links for Code, Issues, Pull requests, Zenhub, Actions (highlighted), Projects, Wiki, Security, Insights, and Settings. A search bar is located on the right. The left sidebar shows the 'Actions' section with a 'New workflow' button and a list of workflow runs. The main area, titled 'All workflows', shows 'Showing runs from all workflows' and a search bar for 'Filter workflow runs'. A single workflow run is displayed with the status 'CI Workflow Done' (indicated by a green checkmark). The run details include 'CI Workflow #1: Commit 139e11c pushed by v0ldgy' and a 'master' branch label. The run completed '1 minute ago' and took '13s'. A 'Sign in now to use Zenhub' button is visible in the bottom right corner.

Event	Status	Branch	Actor
CI Workflow Done	Success	master	v0ldgy